5G Assistant with Structural and Temporal Understanding of Specs

Shuchang Dong*
UCLA
Los Angeles, California, USA
sdong57@ucla.edu

Leica Shen*
UCLA
Los Angeles, California, USA
jshen@cs.ucla.edu

Zijie Zhang*
UCLA
Los Angeles, California, USA
zijiezh@g.ucla.edu

Abstract

3GPP 5G specifications are extensive, highly interconnected, and evolve across releases, making it difficult for engineers to efficiently locate authoritative clauses and understand how requirements change over time. This project presents a practical 5G specification assistant built on top of the DeepSpecs research prototype. We develop a full-stack system that provides a web-based interface for querying specification clauses, exploring version history, and inspecting the structure underlying generated answers. In addition, the system incorporates profiling support to improve transparency into retrieval behavior and system execution. Together, these features enable more accessible, traceable, and interactive exploration of 5G specifications.

Keywords

5G specifications, clause history, version tracking, system profiling, event-bus architecture, LLM integration

1 Introduction

1.1 Project Overview

5G systems are defined by thousands of pages of interdependent 3GPP specifications that evolve across releases. To make this information more accessible, our project develops a 5G Q&A assistant that enables users to query specification clauses, explore their historical changes, and understand how answers are generated.

Our system builds on the DeepSpecs research prototype [1], which provides structured access to specification text and an LLM-driven reasoning engine. However, DeepSpecs offers limited interface support, minimal visibility into retrieval behavior, and no interactive exploration tools.

To bridge this gap, we implemented a practical, release-ready system that adds:

- a web-based UI for searching and browsing specification clauses,
- interactive version-history visualization,
- backend services integrating with the DeepSpecs API, and
- system-profiling instrumentation via an event-bus to surface retrieval latency.

These features provide a transparent and user-friendly environment for interacting with 5G specifications and interpreting DeepSpecs' underlying reasoning steps.

1.2 Problem Statement

Engineers often need to identify specific normative requirements, compare clause revisions between releases, or trace the reasoning behind specification-driven answers. Manually navigating dozens of specification PDFs, cross-references, and historical changes is slow and error-prone.

Key difficulties include:

- **Complex hierarchical documents.** Clause nesting, normative references, and cross-spec interactions are hard to search effectively.
- **Release evolution.** Clause renumbering and incremental updates require nontrivial manual comparison.
- **Lack of traceability in generic AI tools.** Conventional LLMs cannot reliably cite authoritative clauses or reveal their reasoning steps.
- **Limited observability.** Existing systems provide little insight into where retrieval latency occurs or how recursive clause lookups unfold.

Our system addresses these challenges by pairing DeepSpecs' structured retrieval with an interactive interface for clause browsing, history inspection, and system-level profiling. This allows users to understand not only *what* answer the system provides, but also *where it came from* and *how it was retrieved.*

2 System Architecture

2.1 High-Level Design

The system follows a three-tier architecture designed to decouple the user interface from the complex logic of the Retrieval-Augmented Generation (RAG) backend. This separation allows for independent scaling, easier debugging, and the injection of a dedicated profiling layer.

The architecture consists of three primary layers:

(1) **Frontend (Client Layer)**: A React-based Single Page Application (SPA) that manages user interaction, visualization of historical 3GPP clauses, and real-time status polling.
(2) **Middleware (Orchestration & Profiling)**: A Python FastAPI service that acts as a proxy. Its primary role is to measure end-to-end latency, count tokens, and aggregate asynchronous events from the backend to build a complete performance profile.
(3) **Backend (Data & RAG Layer)**: The core engine that executes Q&A system (handling RAG logic). It utilizes an internal `EventBus` to broadcast its internal state (e.g., retrieval steps) to subscribers like the Middleware.

*All authors contributed equally.
Project repository: https://github.com/nams1570/CS211-FQ-2025-Project-5

2.2 Component Details

2.2.1 Frontend: User Experience & Visualization. The Frontend is a **React/TypeScript** SPA that handles user interaction and state management. Its primary architectural role is to decouple visualization from logic. It initiates queries via REST API calls and maintains a responsive UI by polling the Middleware for status updates, ensuring the user is informed of the backend's progress without blocking the interface. It also renders the final performance metrics (latency, tokens) provided by the profiling layer.

2.2.2 Middleware: The Observability Layer. The Middleware acts as an intelligent proxy server built with **FastAPI**. Its main function is to intercept requests to the backend for the purpose of **profiling**. It wraps every query with high-precision timers to measure latency and estimates computational cost by counting tokens. Crucially, it subscribes to the backend's event stream, aggregating granular processing events (like retrieval steps) to provide a complete performance breakdown to the client.

2.2.3 Backend: Event-Driven RAG Engine. The Backend is not just a simple API but an event publisher.

- **EventBus Pattern**: Central to the backend is the `EventBus` component. As the RAG controller executes steps (Retrieval, Ranking, Generation), it publishes events to this bus.
- **Callback Execution**: The bus automatically pushes these events to registered subscribers (the Middleware) via HTTP callbacks. This decoupling allows the backend to focus on logic while the Middleware handles the monitoring.
- **Resource Protection**: It implements rate limiting (per-IP and daily quotas) to prevent abuse of the computationally expensive RAG processes.

2.3 Data Flow

A typical query follows this path:

(1) **User** submits a question via the Frontend.
(2) **Frontend** sends a POST request to Middleware.
(3) **Middleware** starts a timer, forwards the request to the Q&A Backend, and simultaneously begins listening for processing events.
(4) **Q&A Backend** processes the RAG pipeline and returns the answer.
(5) **Middleware** stops the timer, counts the output tokens, updates the rolling averages (EWMA), and returns the enriched response (Answer + Metrics) to the Frontend.
(6) **Frontend** displays the answer and updates the Profiling Dashboard with the new metrics.

3 Frontend Implementation

The frontend is a Single Page Application built with **React**, **TypeScript**, and **Vite**. It serves as the primary interface for users to interact with the 3GPP Q&A system, view clause history, and monitor system performance.

3.1 User Interface Components

The user interface is composed of several modular components located in the `src/components/` directory:

- **Sidebar**: Provides navigation across chat sessions, history views, settings, and the profiling dashboard.
- **ChatArea**: Serves as the main interaction interface, displaying user queries and system responses in a conversational format.
- **ClauseHistory**: Visualizes the evolution of 3GPP clauses across different specification versions.
- **Profiling**: Displays performance metrics such as request latency and token usage, along with their rolling averages.

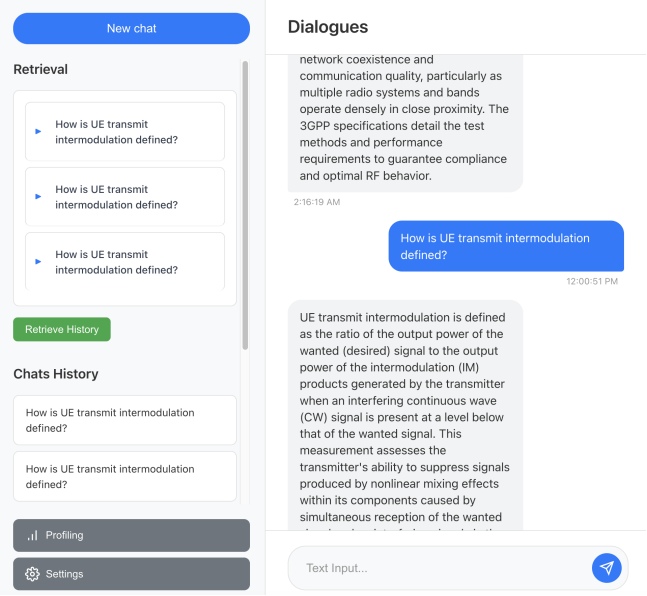

Figure 1: User Interface Overview. The sidebar (left) allows navigation, while the main area (right) displays the chatbox.

3.2 State Management and API Integration

The frontend manages application state to support multi-turn conversations and performance visualization. Core state includes active chat sessions, message history, and profiling data returned from the middleware.

Communication with backend services is encapsulated in a lightweight API layer that issues requests for question answering and event polling. This abstraction decouples frontend logic from backend endpoints and allows the interface to update responses and profiling metrics asynchronously as query execution progresses.

3.3 Clause History Visualization

The Clause History feature allows users to track how a specific regulation has changed over time. When a user selects Clause History from a retrieved context, the frontend queries the changeDB service.

The ClauseHistory component organizes the data hierarchically:

(1) **Document Level**: Groups versions by 'docID' (e.g., 38.211).
(2) **Version Level**: Shows transitions between specific versions (e.g., v15.2.0 → v15.3.0) and the timestamps of those changes.

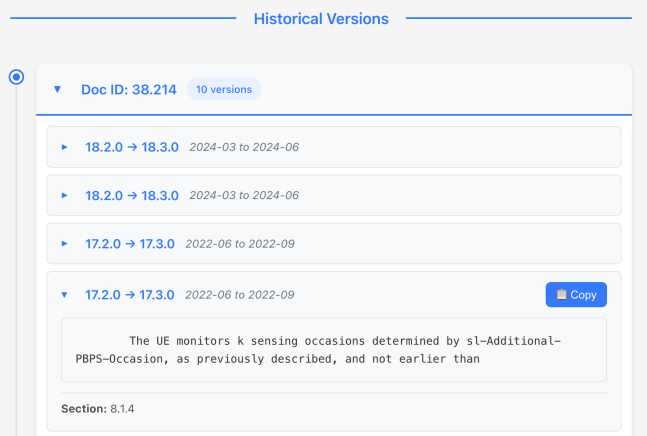

Figure 2: Clause History Visualization. This interface displays the evolution of a specific 3GPP clause (e.g., Section 8.1.4).

3.4 Real-time Feedback System

To improve usability during long-running queries, the frontend provides real-time feedback on backend execution progress. While a query is being processed, the interface displays intermediate status updates corresponding to different execution stages.

These updates are obtained by periodically querying the middleware for execution events associated with the current request. As events arrive, the frontend updates the interface to reflect the system's progress, allowing users to understand that the query is actively being processed rather than stalled.

4 Middleware & Profiling

The middleware layer sits between the frontend and the DeepSpecs backend and is primarily responsible for system profiling. It forwards user queries to the backend while measuring latency, estimating token usage, and collecting execution events.

4.1 Profiling Mechanism

The profiling logic is implemented at the middleware entry point, which wraps all requests sent from the frontend to the backend. Instead of embedding profiling code inside the RAG pipeline, all measurement logic is placed at this boundary layer.

When a user submits a query, the middleware receives the request, assigns a unique request identifier, and records a start timestamp. The request is then forwarded to the backend without modification. After the backend finishes processing the query and returns the final response, the middleware records an end timestamp and computes the end-to-end latency.

Beyond total latency, the middleware also associates profiling information with individual queries using the request identifier. This enables latency to be attributed to multiple retrieval stages executed as part of a single query, as well as the collection of token usage for the generated answer. Rolling averages over recent queries are computed to capture longer-term performance behavior.

Overall, this design keeps profiling centralized and lightweight while allowing the system to observe query execution in a structured and consistent manner.

4.2 EventBus Architecture

Profiling is implemented using an EventBus that coordinates internal execution events and routes profiling information to interested components. During query processing, the backend emits structured events as it progresses through different execution stages. Each event contains a type, a timestamp, and associated metadata.

The EventBus organizes events on a per-query basis. It maintains a short history of events for each query and tracks which subscribers are registered for that query. When a new event arrives, the EventBus matches it with the corresponding subscribers and delivers the event through callback mechanisms. Subscribers then extract relevant profiling information and forward it to the frontend.

The EventBus is implemented using a publish–subscribe model. This design decouples event producers from consumers and allows execution events to be delivered to multiple subscribers without modifying backend logic. In the current system, the frontend acts as the primary subscriber, while the same mechanism naturally supports additional subscribers in future extensions.

4.3 Profiling Metrics

The middleware collects profiling information for each query along two dimensions: latency and token usage. These metrics provide visibility into both overall system behavior and individual retrieval stages.

- **Latency Tracking**: For each request, the middleware records a start timestamp before forwarding the query to the backend and an end timestamp after receiving the final response, which together determine the end-to-end latency. In addition, the middleware records timestamps for three retrieval stages: initial specification retrieval, initial technical document retrieval, and secondary retrieval, allowing stage-level latency analysis.
- **Token Counting**: Token usage is estimated based on the generated response text. This metric provides a consistent measure for comparing queries and understanding relative computational cost.
- **Data Persistence**: Profiling results are stored persistently across requests, including per-query latency and token counts. The system also maintains rolling averages using an exponentially weighted moving average (EWMA) to smooth short-term variance and track long-term performance trends.

4.4 Profiling Dashboard

The profiling dashboard provides a visual summary of performance metrics collected by the middleware. It presents both per-request statistics and rolling averages to help users understand system behavior at a glance. Figure 3 shows the profiling dashboard.

The dashboard is divided into two main sections: latency metrics and token metrics. For latency, it displays the execution time of recent requests as well as the rolling average latency, highlighting

performance variability across queries. For token usage, it shows the number of tokens generated per request along with the rolling average token count.

By combining per-request values with aggregated statistics, the dashboard allows users to quickly identify slow queries, observe long-term trends, and correlate latency with token usage.

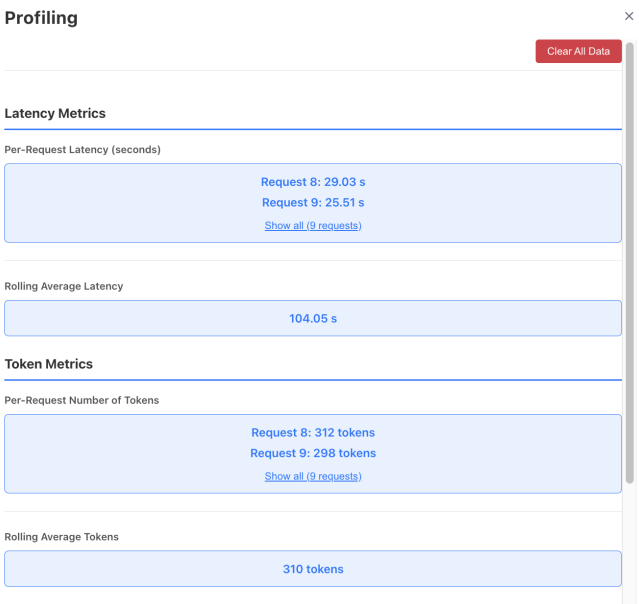

Figure 3: Profiling Dashboard. The dashboard visualizes per-request latency and token usage, along with their rolling averages.

5 Performance Evaluation

To validate the efficiency and observability of the system, we conducted a series of query tests and analyzed the metrics captured by our profiling middleware. The data provides insights into the latency bottlenecks and resource consumption of the RAG pipeline.

5.1 Latency Analysis

We observed significant variability in end-to-end response times across 15 test queries, as shown in Table 1.

Key Observations:

- **High Variance**: The system exhibits a wide latency distribution, with some queries completing in **25-30 seconds** while others exceed **180 seconds**.
- **Query Complexity**: Since the system does not utilize caching, this disparity is directly attributable to the complexity of the retrieval path required for each specific question. "Easy" questions are likely resolved during the *Initial Spec Retrieval*, whereas "Hard" questions force the pipeline into the expensive *Secondary Retrieval* stage to find sufficient context.
- **Optimization Potential**: The data suggests that reducing the frequency or duration of the Secondary Retrieval phase—perhaps by improving the precision of the initial search—would be the most effective way to stabilize performance and reduce the incidence of >3 minute wait times.

Table 1: Profiling Data: Latency and Token Usage (Last 15 Requests)

Request ID	Latency (seconds)	Answer Tokens
1	180.88	318
2	187.82	347
3	194.98	361
4	203.83	345
5	25.96	282
6	24.46	259
7	196.87	321
8	29.03	312
9	25.51	298
10	33.57	304
11	31.76	344
12	27.47	313
13	25.42	287
14	28.34	313
15	28.89	296
Average	**76.25 s**	**313.3**

5.2 Token Consumption

While latency exhibited high variance, token usage remained remarkably stable across all 15 test queries, as detailed in Table 1.

The system generated an average of **313.3 tokens** per answer, with values ranging from a minimum of 259 to a maximum of 361 tokens. This relatively narrow distribution indicates that the Generation phase of the pipeline is consistent; the RAG model adheres well to the system prompt's instructions regarding answer length and depth, regardless of whether the retrieval took 25 seconds or 200 seconds.

This predictability is crucial for cost estimation. Since LLM API costs are typically token-based, we can reliably forecast the operational expense of the system to be approximately constant per query, independent of the retrieval complexity.

5.3 Stage-wise Breakdown

The most critical insight comes from the granular stage duration analysis presented in Table 2, which breaks down the total latency into specific RAG components.

Table 2: Retrieval Stage Durations

Stage	Current (s)	Avg (s)
Init Spec Retrieval	0.57	0.73
Init Tdoc Retrieval	4.84	5.70
Secondary Retrieval	5.75	24.86

The data highlights **Secondary Retrieval** as the primary bottleneck:

- **Initial Spec Retrieval**: Extremely fast (~0.57s), indicating efficient indexing for high-level specification lookups.

- **Initial Tdoc Retrieval**: Moderate speed (~4.84s).
- **Secondary Retrieval**: The rolling average is **24.86s**, making it the most expensive operation. This stage likely involves complex cross-referencing or iterative searches that could be optimized in future work.

5.4 Latency bottlenecks

The profiling results indicate that the primary source of latency variability lies in the secondary retrieval phase. In particular, queries that trigger secondary retrieval exhibit significantly longer response times.

Secondary retrieval involves deeper cross-referencing and iterative clause resolution across 3GPP specifications. This process often requires multiple dependent lookups to assemble sufficient contextual information, which substantially increases execution time. Consequently, whether a query triggers secondary retrieval largely determines whether it completes within tens of seconds or extends to several minutes.

These observations suggest that latency bottlenecks are primarily structural rather than computational. Potential improvements include increasing the precision of early-stage retrieval to avoid unnecessary secondary expansion, introducing caching for frequently accessed clause structures, or pruning redundant reference chains. Such optimizations could reduce tail latency and improve overall performance stability.

5.5 Conclusion on Performance

Overall, the performance evaluation shows that the system exhibits stable token usage but highly variable latency. Token counts remain consistent across queries, indicating predictable generation behavior and relatively stable computational cost.

In contrast, end-to-end latency varies widely depending on retrieval complexity. Profiling reveals that this variability is driven mainly by the secondary retrieval stage. While the current design prioritizes correctness and comprehensive context retrieval, these results highlight clear opportunities for optimization.

Importantly, the profiling infrastructure provides sufficient visibility to identify these bottlenecks and guide future improvements. By exposing stage-level latency and long-term trends, the system enables data-driven optimization without altering the core reasoning pipeline.

6 Challenges and Solutions

6.1 CORS Issue

When integrating the frontend with the DeepSpecs Q&A backend, we encountered a browser-level Cross-Origin Resource Sharing (CORS) restriction. All API requests functioned correctly in Postman, but failed immediately when issued from the browser. This discrepancy occurred because CORS is a security mechanism enforced only by browsers; tools like Postman do not apply these checks.

CORS blocks a webpage on one origin from making requests to a different origin unless the backend explicitly authorizes the request by returning an `Access-Control-Allow-Origin` header. Since our backend did not include this header, the browser rejected the request before it ever reached the server, even though the backend itself was functioning correctly.

To enable development, we used Vite's development proxy, which runs under the same origin as the frontend and transparently forwards requests to the backend. From the browser's perspective, all requests appear same-origin, so the CORS check is bypassed entirely. This allowed us to continue development without modifying the backend configuration.

However, this workaround is not a long-term production solution. Vite's proxy only applies to local development. A robust deployment would require either enabling proper CORS headers on the backend or introducing a production-grade reverse proxy—such as Nginx, Traefik, or an API gateway—that uniformly manages cross-origin traffic. Migrating to a more reliable proxy remains part of our future work.

7 Future Work

There are several directions in which the system can evolve, including system optimization and latency reduction, improved scalability, security hardening, historical diff visualization, and expanded backend controls. During development and the Q&A discussion, we identified several areas that would meaningfully extend the system in future iterations. We discuss these in more detail below.

7.1 DeepSpecs Optimizations

DeepSpecs [1] resolves clause-level cross-references by recursively retrieving referenced clauses and assembling the dependency structure required to answer technical questions. This design is necessary because 3GPP specifications are highly modular, with definitions and conditions frequently distributed across multiple clauses and documents.

Profiling results show that most end-to-end latency arises from this recursive reference resolution stage. Resolving references involves repeated metadata-guided lookups and re-ranking steps, and queries that touch densely cross-referenced specification regions naturally incur deeper traversal and longer runtimes.

Although DeepSpecs primarily emphasizes correctness and structural grounding rather than runtime efficiency, several optimizations could improve responsiveness in a production setting without changing retrieval semantics:

- **Caching resolved reference chains** to avoid redundant lookups for frequently accessed specification regions.
- **Incremental or streaming retrieval** to reduce perceived latency by exposing partial context earlier.
- **Parallel retrieval of independent references** to shorten wall-clock time for multi-branch resolution.
- **Precomputing a clause-level reference graph** using SpecDB and ChangeDB metadata [1] to accelerate traversal.

These optimizations are exploratory rather than prescriptive. While the current recursive design is essential for accuracy, profiling highlights clear opportunities to improve responsiveness and scalability in interactive settings.

7.2 User-Provided Specification Integration

During the Q&A discussion, it was suggested that supporting user-provided specification documents could further extend the system's

flexibility. Such capability would allow practitioners to incorporate additional materials—such as vendor-specific guidelines, operator documentation, or supplementary 3GPP releases—into the question-answering workflow.

A future extension of the system could introduce an ingestion pathway that:

- extracts clause-like structures from uploaded documents,
- aligns them to the metadata schema used by SpecDB, and
- integrates them into the retrieval process in a controlled and transparent manner.

This feature would enable the assistant to operate over a broader and more customizable document set, supporting use cases where organizations rely on internal or proprietary technical standards alongside 3GPP specifications.

7.3 Interactive Clause History and Diff Queries

Two complementary extensions can further improve how the system exposes specification evolution. First, our current system retrieves version-aware clause content through DeepSpecs but does not yet provide a dedicated visual diff interface. A logical next step is to incorporate a diff-style viewer that highlights added, removed, and modified lines across releases. Such a tool would help practitioners study version changes more efficiently and align with workflows commonly used in code review and standard analysis.

Second, during the Q&A session, it was suggested that the assistant could directly answer natural-language questions about clause evolution, such as "How did Clause 6.3 change between Release 16 and Release 18?" or "What was removed in the latest revision?" Supporting these interactions would require mapping user queries to evolution-aware retrieval operations and returning structured summaries of differences or change traces.

Future versions of the system could enable these capabilities by:

- exposing evolution information through additional API endpoints,
- extending backend logic to interpret diff- and history-oriented queries, and
- enhancing the UI to present version-to-version differences interactively.

Together, these extensions would strengthen the system's usefulness for engineers who routinely inspect specification changes and track the evolution of technical features across 3GPP releases.

8 Conclusion

In this project, we developed a practical 5G specification assistant that extends the DeepSpecs research prototype into a user-facing, release-ready system. By integrating a web-based interface with structured backend services, our system enables users to query 3GPP specifications, explore clause version history, and inspect the structure and provenance underlying generated answers.

Beyond basic question answering, the system emphasizes transparency and usability. The frontend provides interactive clause browsing and history views, while the middleware introduces profiling support that exposes system execution stages and latency behavior. Together, these components support both engineers seeking authoritative answers and developers interested in understanding system performance and retrieval behavior.

Overall, this project demonstrates how research-oriented specification reasoning can be transformed into a usable tool for standards exploration and analysis. The resulting system provides a foundation for future extensions that further enhance performance, extensibility, and evolution-aware interaction with complex technical specifications.

References

[1] A. Ganapathy Manvattira, Y. Xu, Z. Dang, and S. Lu. Deepspecs: Expert-level question answering in 5g. *arXiv preprint arXiv:2511.01305*, 2025.
[2] Meta Platforms, Inc. React documentation. https://react.dev/, 2024. Accessed: 2025-01-10.
[3] Sebastián Ramírez. Fastapi documentation. https://fastapi.tiangolo.com/, 2024. Accessed: 2025-01-10.
[4] 3rd Generation Partnership Project (3GPP). 3gpp specification standards. https://www.3gpp.org/specifications, 2025. Accessed: 2025-01-10.